57. Строки (std::string)

Вашей первой программой на языке C++, вероятно, была всеми известная программа «Hello, world!»:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

Не так ли? Но что такое `Hello, world!` ? `Hello, world!` — это последовательность символов или просто **строка** (англ. **«string»**). В языке C++ мы используем строки для представления текста (имен, адресов, слов и предложений). Строковые литералы (такие как `Hello, world!`) помещаются в двойные кавычки.

Поскольку их часто используют в программах, то большинство современных языков программирования имеют встроенный тип данных string. В языке C++ есть также этот тип, но не как часть основного языка, а как часть Стандартной библиотеки C++.

Оглавление:

1. Тип данных string
2. Ввод/вывод строк
3. Использование std::getline()
4. Использование std::getline() с std::cin
5. Добавление строк
6. Длина строк
7. Тест

Тип данных string

Чтобы иметь возможность использовать строки в C++, сначала нужно подключить **заголовочный файл** string. Как только это будет сделано, мы сможем определять переменные типа string:

```
#include <string>

// ...
std::string name;
```

Как и с обычными переменными, мы можем инициализировать переменные типа string или присваивать им значения:

```
std::string name{"Sasha"}; // инициализируем переменную name с помощью строки "Sasha"
name = "Masha"; // присваиваем переменной name строковый литерал "Masha"
```

Строки также могут содержать числа:

```
std::string myID{"34"}; // "34" здесь - это не число, это 2 символа
```

Стоит отметить, что присваиваемые числа тип string обрабатывает как текст, а не как числа, и, следовательно, ими нельзя манипулировать как обычными числами (например, вы не сможете выполнить с ними **арифметические операции**). Язык C++ **автоматически не преобразовывает** их в значения целочисленных типов или типов с плавающей точкой.

Ввод/вывод строк

Строки можно выводить с помощью std::cout:

```
#include <iostream>
#include <string>

int main()
{
    std::string name{"Sasha"};
    std::cout << "My name is " << name;

    return 0;
}
```

Результат выполнения программы:

```
My name is Sasha
```

А вот с std::cin дела обстоят несколько иначе. Рассмотрим следующий пример:

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Enter your full name: ";
    std::string myName;
    std::cin >> myName; // это будет работать не так, как ожидается, поскольку извлечение данных из потока std::cin остановится

    std::cout << "Enter your age: ";
    std::string myAge;
    std::cin >> myAge;

    std::cout << "Your name is " << myName << " and your age is " << myAge;
}
```

Результат выполнения программы:

```
Enter your full name: Sasha Mak
Enter your age: Your name is Sasha and your age is Mak
```

Хм, что-то не так! Что же случилось? Оказывается, оператор извлечения (`>>`) возвращает символы из входного потока данных только до первого пробела. Все остальные символы остаются внутри cin, ожидая следующего извлечения.

Поэтому, когда мы использовали оператор `>>` для извлечения данных в переменную `myName` , только `Sasha` был извлечен, `Mak` остался внутри std::cin, ожидая следующего извлечения. Когда мы использовали оператор `>>` снова, чтобы извлечь данные в переменную `myAge` , мы получили `Mak` вместо `25` . Если бы мы сделали третье извлечение, то получили бы `25` .

Использование std::getline()

Чтобы извлечь полную строку из входного потока данных (вместе с пробелами), используйте **функцию std::getline()**. Она принимает два параметра: первый — std::cin, второй — переменная типа string.

Вот программа, приведенная выше, но уже с использованием std::getline():

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Enter your full name: ";
    std::string myName;
    std::getline(std::cin, myName); // полностью извлекаем строку в переменную myName

    std::cout << "Enter your age: ";
    std::string myAge;
    std::getline(std::cin, myAge); // полностью извлекаем строку в переменную myAge

    std::cout << "Your name is " << myName << " and your age is " << myAge;
}
```

Теперь работает как надо:

```
Enter your full name: Sasha Mak
Enter your age: 25
Your name is Sasha Mak and your age is 25
```

Использование std::getline() с std::cin

Извлечение данных из std::cin с помощью std::getline() иногда может приводить к неожиданным результатам. Например, рассмотрим следующую программу:

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Pick 1 or 2: ";
    int choice;
    std::cin >> choice;

    std::cout << "Now enter your name: ";
    std::string myName;
    std::getline(std::cin, myName);

    std::cout << "Hello, " << myName << ", you picked " << choice << '\n';

    return 0;
}
```

Возможно, вы удивитесь, но когда вы запустите эту программу, и она попросит вас ввести ваше имя, она не будет ожидать вашего ввода, а сразу выведет результат (просто пробел вместо вашего имени)!

Пробный запуск программы:

```
Pick 1 or 2: 2
Now enter your name: Hello, , you picked 2
```

Почему так? Оказывается, когда вы вводите числовое значение, поток cin захватывает вместе с вашим числом и символ новой строки. Поэтому, когда мы ввели 2, cin фактически получил `2\n` . Затем он извлек значение `2` в переменную, оставляя `\n` (символ новой строки) во входном потоке. Затем, когда std::getline() извлекает данные для `myName` , он видит в потоке `\n` и думает, что мы, должно быть, ввели просто пустую строку! А это определенно не то, что мы хотим.

Хорошей практикой является удалять из входного потока данных символ новой строки. Это можно сделать следующим образом:

```
std::cin.ignore(32767, '\n'); // игнорируем символы вплоть до следующего '\n', но не более чем 32767 символов
```

Если мы вставим эту строку непосредственно после получения входных данных, то символ новой строки будет удален из входного потока, и программа будет работать должным образом:

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Pick 1 or 2: ";
    int choice;
    std::cin >> choice;

    std::cin.ignore(32767, '\n'); // удаляем символ новой строки из входного потока данных

    std::cout << "Now enter your name: ";
    std::string myName;
    std::getline(std::cin, myName);

    std::cout << "Hello, " << myName << ", you picked " << choice << '\n';

    return 0;
}
```

Правило: При вводе числовых значений не забывайте удалять символ новой строки из входного потока данных с помощью std::cin.ignore().

Добавление строк

Вы можете использовать оператор `+` для объединения двух строк или оператор `+=` для добавления одной строки к другой.

В следующей программе мы протестируем эти два оператора, а также покажем, что произойдет, если вы попытаетесь использовать оператор `+` для соединения двух числовых строк:

```
#include <iostream>
#include <string>

int main()
{
    std::string x("44");
    std::string y("12");

    std::cout << x + y << "\n"; // объединяем строки x и y (а не складываем числа)
    x += " cats";
    std::cout << x;

    return 0;
}
```

Результат выполнения программы:

```
4412
44 cats
```

Обратите внимание, оператор `+` объединил две числовые строки в одну (`44 + 12 = 4412`). Он не складывал эти строки как числа.

Длина строк

Чтобы узнать длину строки, мы можем сделать следующее:

```
#include <iostream>
#include <string>

int main()

{
    std::string myName("Sasha");
    std::cout << myName << " has " << myName.length() << " characters\n";
    return 0;
}
```

Результат выполнения программы:

```
Sasha has 5 characters
```

Обратите внимание, вместо запроса длины строки как `length(myName)` , мы пишем `myName.length()` .

Функция запроса длины строки не является обычной функцией как те, которые мы использовали ранее. Это особый тип функции класса std::string, который называется методом. Мы поговорим об этом детально, когда будем рассматривать классы.

Тест

Напишите программу, которая просит у пользователя ввести его имя, фамилию и возраст. В результате, укажите пользователю, сколько лет он прожил на каждую букву из его имени и фамилии (чтобы было проще, учитывайте также пробелы). Например:

```
Enter your full name: Tom Cats
Enter your age: 45
You've lived 5.625 years for each letter in your name.
```

> ***Уточнение:*** Возраст 45 делится на длину имени и фамилии «Tom Cats» (8 букв, учитывая пробел), что равно 5.625.

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Enter your full name: ";
    std::string myName;
    std::getline(std::cin, myName); // извлекаем целую строку из входного потока в переменную myName

    std::cout << "Enter your age: ";
    int myAge; // переменные myAge должны быть типа int, а не типа string, чтобы мы могли производить с ней арифметические опера
    std::cin >> myAge;

    int letters = myName.length(); // вычисляем длину переменной myName (учитывая пробелы)
    double agePerLetter = static_cast<double>(myAge) / letters; // используем оператор static_cast, чтобы изменить тип перемен
    std::cout << "You've lived " << agePerLetter << " years for each letter in your name.\n";

    return 0;
}
```